

Datos de la Práctica 3

Práctica 3 de 4:	<i>Perceptron</i>
Objetivo de la práctica:	Aplicar los conocimientos en redes neuronales y el lenguaje de programación Python, para programar un <i>perceptron</i> , a través del desarrollo de una aplicación.
Temas y subtemas asociados:	5. Redes neuronales 5.2. Control con redes neuronales 5.2.1. Red de perceptron 5.2.2. Red de retropropagación
Fecha:	
Duración (horas):	2 horas
Laboratorio de:	Cómputo
Equipo de seguridad para ingresar al laboratorio (indispensable):	N/A
Software requerido:	Cualquier IDE que soporte desarrollo con Python 3
Equipo necesario en laboratorio:	Computadora
Material/Sustancias/Reactivos disponible en laboratorio:	N/A
Material/Sustancias/Reactivos aportado por el alumno:	N/A

Desarrollo práctica 3:

Indicaciones de la práctica:

- Implementar una matriz en Python para los siguientes datos de entrada:
 - 0,0,1
 - 1,1,1
 - 1,0,1
 - 0,1,1
- A su vez se deberá implementar una segunda matriz la cual funcionará como los datos de salida esperada para el *perceptron*, los datos de la salida esperada son:
 - 0
 - 1
 - 1
 - 0

- Crear una función la cual representará a la función sigmoideal. Recuerda que la fórmula es:

$$\phi(x) = \frac{1}{1 + e^{-x}}$$

- Crea una matriz con los pesos iniciales para las 4 entradas. Estos valores deberán ser valores aleatorios.

Desarrollo práctica 3:

5. Realiza la propagación hacia adelante, realizando el producto punto entre la matriz con las entradas y la matriz con los pesos.
6. Aplica la función sigmoideal programada en el paso 3, al resultado obtenido en el paso 5.
7. Crear una función la cual representará a la gradiente de la curva sigmoideal. Para ello recuerda que la fórmula es:

$$\phi'(x) = x \cdot (1 - x)$$

8. Calcula el error del resultado obtenido en la propagación hacia adelante, para ello tienes a la siguiente fórmula:

$$\text{error} = \text{salida esperada} - \text{salida obtenida}$$

9. Posteriormente se calcularán los ajustes en base a la siguiente fórmula:

$$f(x) = \text{error} * \text{gradiente de la curva sigmoideal de la salida}$$

10. Recalcula los pesos realizando el producto punto entre la matriz que contiene las entradas y la matriz que contiene los ajustes
11. Incluye el proceso de propagación adelante y propagación atrás en un ciclo que se repita 50,000 veces.
12. Imprime en pantalla la salida obtenida y el valor de los pesos después del proceso de entrenamiento.

Resultados obtenidos:

Desarrollar aplicaciones usando un *perceptron* con retropropagación, mediante del desarrollo de una aplicación en Python.

Evidencia de la práctica: (fotografías, videos, archivo, etc.)

Entrega del código fuente (archivo .py)

Conclusiones:

Al término de la práctica se espera que el alumno sea capaz de comprender como se pueden desarrollar y aplicar las redes neuronales para la resolución de problemas usando la inteligencia artificial.

Bibliografía:

Ponce, J., Soto, A., Sayuri, F. (2014). Inteligencia Artificial. LATIn Project.

Criterios de evaluación:

En el código fuente:

- El programa realiza la propagación hacia adelante sin errores.
- El programa realiza la propagación hacia atrás sin errores.
- El programa calcula de forma correcta los pesos después del proceso de entrenamiento.